



Data Science Applications

In this chapter, we shall study a few applications of linear programming to an area of statistics called regression and to an area of machine learning called support vector machines. As a specific example for our study of regression, we shall use size and iteration-count data collected from a standard suite of linear programming problems to derive a regression estimate of the number of iterations needed to solve problems of a given size. And, as an example of a support vector machine, we will look at the modern day problem of using computers to understand/decipher handwritten messages. This application is called optical character recognition (OCR).

1. Measures of Mediocrity

We begin our discussion with an example. Here are the midterm exam scores for a linear programming course:

28, 62, 80, 84, 86, 86, 92, 95, 98.

Let m denote the number of exam scores (i.e., $m = 9$) and let b_i , $i = 1, 2, \dots, m$, denote the actual scores (arranged in increasing order as above). The most naive measure of the “average” score is just the *mean* value, $\bar{x}$, defined by

$$\bar{x} = \frac{1}{m} \sum_{i=1}^m b_i = 79.0.$$

This is an example of a statistic, which, by definition, is a function of a set of data. Statistics are computed so that one does not need to bother with reporting large tables of raw numbers. (Admittedly, the task of reporting the above list of 9 exam scores is not very onerous, but this is just an example.) Now, suppose the professor in question did not report the scores but instead just gave summary statistics. Consider the student who got an 80 on the exam. This student surely did not feel great about this score but might have thought that at least it was better than average. However, as the raw data makes clear, this student really did worse than average¹ on the exam (the professor

¹“Average” is usually taken as synonymous with “mean” but in this section we shall use it in an imprecise sense, employing other technically defined terms for specific meanings.

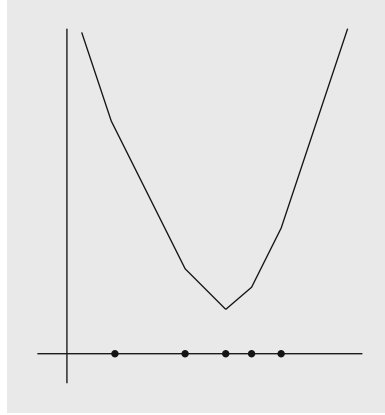


FIGURE 12.1. The objective function whose minimum occurs at the median.

confesses that the exam was rather easy). In fact, out of the nine students, the one who got an 80 scored third from the bottom of the class. Furthermore, the student who scored worst on the exam did so badly that one might expect this student to drop the course, thereby making the 80 look even worse.

Any statistician would, of course, immediately suggest that we report the median score instead of the mean. The *median* score is, by definition, that score which is worse than half of the other scores and better than the other half. In other words, the median $\hat{x}$ is defined as

$$\hat{x} = b_{(m+1)/2} = 86.$$

(Here and in various places in this chapter, we shall assume that m is odd so that certain formulas such as this one remain fairly simple.) Clearly, the 86 gives a more accurate indication of what the average score on the exam was.

There is a close connection between these statistical concepts and optimization. For example, the mean $\bar{x}$ minimizes, over all real numbers x , the sum of the squared deviations between the data points and x itself. That is,

$$\bar{x} = \operatorname{argmin}_{x \in \mathbb{R}} \sum_{i=1}^m (x - b_i)^2.$$

To verify this claim, we let $f(x) = \sum_{i=1}^m (x - b_i)^2$, differentiate with respect to x , and set the derivative to zero to get

$$f'(x) = \sum_{i=1}^m 2(x - b_i) = 0.$$

Solving this equation for the critical point² x , we see that

$$x = \frac{1}{m} \sum_{i=1}^m b_i = \bar{x}.$$

The fact that this critical point is a minimum rather than a maximum (or a saddle point) follows from the fact that $f''(x) > 0$ for all $x \in \mathbb{R}$.

The median $\hat{x}$ also enjoys a close connection with optimization. Indeed, it is the point that minimizes the sum of the absolute values of the difference between each data point and itself. That is,

$$\hat{x} = \operatorname{argmin}_{x \in \mathbb{R}} \sum_{i=1}^m |x - b_i|.$$

To see that this is correct, we again use calculus. Let

$$f(x) = \sum_{i=1}^m |x - b_i|.$$

This function is continuous, piecewise linear, and convex (see Figure 12.1). However, it is not differentiable at the data points. Nonetheless, we can look at its derivative at other points to see where it jumps across zero. The derivative, for $x \notin \{b_1, b_2, \dots, b_m\}$, is

$$f'(x) = \sum_{i=1}^m \operatorname{sgn}(x - b_i),$$

where

$$\operatorname{sgn}(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x = 0 \\ -1 & \text{if } x < 0. \end{cases}$$

Hence, we see that the derivative at x is just the number of data points to the left of x minus the number of data points to the right. Clearly, this derivative jumps across zero at the median, implying that the median is the minimum.

In this chapter, we shall discuss certain generalizations of means and medians called regressions. At the end, we will consider a specific example that is of particular interest to us: the empirical average performance of the simplex method.

2. Multidimensional Measures: Regression Analysis

The analysis of the previous section can be recast as follows. Given a “random” observation b , we assume that it consists of two parts: a fixed, but unknown, part denoted by x and a random fluctuation about this fixed part, which we denote by ϵ . Hence,

²Recall from calculus that a *critical point* is any point at which the derivative vanishes or fails to exist.

$$b = x + \epsilon.$$

Now, if we take several observations and index them as $i = 1, 2, \dots, m$, the b 's and the ϵ 's will vary, but x is assumed to be the same for all observations. Therefore, we can summarize the situation by writing

$$b_i = x + \epsilon_i, \quad i = 1, 2, \dots, m.$$

We now see that the mean is simply the value of x that minimizes the sum of the squares of the ϵ_i 's. Similarly, the median is the value of x that minimizes the sum of the absolute values of the ϵ_i 's.

Sometimes one wishes to do more than merely identify some sort of “average.” For example, a medical researcher might collect blood pressure data on thousands of patients with the aim of identifying how blood pressure depends on age, obesity (defined as weight over height), sex, etc. So associated with each observation b of a blood pressure are values of these *control* variables. Let us denote by a_1 the age of a person, a_2 the obesity, a_3 the sex, etc. Let n denote the number of different control variables being considered by the researcher. In (linear) regression analysis, we assume that the *response* b depends linearly on the control variables. Hence, we assume that there are (unknown) numbers x_j , $j = 1, 2, \dots, n$, such that

$$b = \sum_{j=1}^n a_j x_j + \epsilon.$$

This equation is referred to as the *regression model*. Of course, the researcher collects data from thousands of patients, and so the data items, b and the a_j 's, must be indexed over these patients. That is,

$$b_i = \sum_{j=1}^n a_{ij} x_j + \epsilon_i, \quad i = 1, 2, \dots, m.$$

If we let b denote the vector of observations, ϵ the vector of random fluctuations, and A the matrix whose i th row consists of the values of the control variables for the i th patient, then the regression model can be expressed in matrix notation as

$$(12.1) \quad b = Ax + \epsilon.$$

In regression analysis, the goal is to find the vector x that best explains the observations b . Hence, we wish to pick values that minimize, in some sense, the vector ϵ 's. Just as for the mean and median, we can consider minimizing either the sum of the squares of the ϵ_i 's or the sum of the absolute values of the ϵ_i 's. There are even other possibilities. In the next two sections, we will discuss the range of possibilities and then give specifics for the two mentioned above.

3. L^2 -Regression

There are several notions of the size of a vector. The most familiar one is the Euclidean length

$$\|y\|_2 = \left(\sum_i y_i^2 \right)^{1/2}.$$

This notion of length corresponds to our physical notion (at least when the dimension is low, such as 1, 2, or 3). However, one can use any power inside the sum as long as the corresponding root accompanies it on the outside of the sum. For $1 \leq p < \infty$, we get then the so-called L^p -norm of a vector y

$$\|y\|_p = \left(\sum_i y_i^p \right)^{1/p}.$$

Other than $p = 2$, the second most important case is $p = 1$ (and the third most important case corresponds to the limit as p tends to infinity).

Measuring the size of ϵ in (12.1) using the L^2 -norm, we arrive at the L^2 -regression problem, which is to find $\bar{x}$ that attains the minimum L^2 -norm for the difference between b and Ax . Of course, it is entirely equivalent to minimize the square of the L^2 -norm, and so we get

$$\bar{x} = \operatorname{argmin}_x \|b - Ax\|_2^2.$$

Just as for the mean, there is an explicit formula for $\bar{x}$. To find it, we again rely on elementary calculus. Indeed, let

$$f(x) = \|b - Ax\|_2^2 = \sum_i \left(b_i - \sum_j a_{ij}x_j \right)^2.$$

In this multidimensional setting, a critical point is defined as a point at which the derivative with respect to every variable vanishes. So if we denote a critical point by $\bar{x}$, we see that it must satisfy the following equations:

$$\frac{\partial f}{\partial x_k}(\bar{x}) = \sum_i 2 \left(b_i - \sum_j a_{ij}\bar{x}_j \right) (-a_{ik}) = 0, \quad k = 1, 2, \dots, n.$$

Simplifying these equations, we get

$$\sum_i a_{ik}b_i = \sum_i \sum_j a_{ik}a_{ij}\bar{x}_j, \quad k = 1, 2, \dots, n.$$

In matrix notation, these equations can be summarized as follows:

$$A^T b = A^T A \bar{x}.$$

In other words, assuming that $A^T A$ is invertible, we get

$$(12.2) \quad \bar{x} = (A^T A)^{-1} A^T b.$$

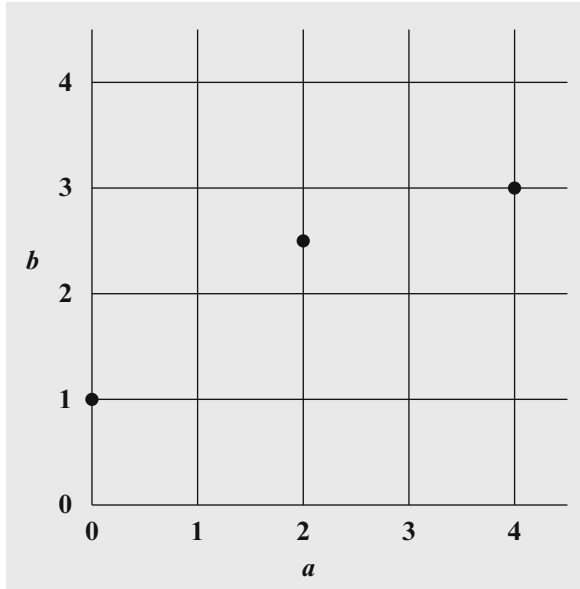


FIGURE 12.2. Three data points for a linear regression.

This is the formula for L^2 -regression. It is also commonly called *least squares regression*. In Section 12.6, we will use this formula to solve a specific regression problem.

Example. The simplest and most common regression model arises when one wishes to describe a response variable b as a linear function of a single input variable a . In this case, the model is

$$b = ax_1 + x_2.$$

The unknowns here are the slope x_1 and the intercept x_2 . Figure 12.2 shows a plot of three pairs (a, b) through which we want to draw the “best” straight line. At first glance, this model does not seem to fit the regression paradigm, since regression models (as we have defined them) do not involve a term for a nonzero intercept. But the model here can be made to fit by introducing a new control variable, say, a_2 , which is always set to 1. While we are at it, let us change our notation for a to a_1 so that the model can now be written as

$$b = a_1x_1 + a_2x_2.$$

The three data points can then be summarized in matrix notation as

$$\begin{bmatrix} 1 \\ 2.5 \\ 3 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 2 & 1 \\ 4 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} \epsilon_1 \\ \epsilon_2 \\ \epsilon_3 \end{bmatrix}.$$

For this problem,

$$A^T A = \begin{bmatrix} 0 & 2 & 4 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 2 & 1 \\ 4 & 1 \end{bmatrix} = \begin{bmatrix} 20 & 6 \\ 6 & 3 \end{bmatrix}$$

and

$$A^T b = \begin{bmatrix} 0 & 2 & 4 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 2.5 \\ 3 \end{bmatrix} = \begin{bmatrix} 17 \\ 6.5 \end{bmatrix}.$$

Hence,

$$\bar{x} = \begin{bmatrix} \bar{x}_1 \\ \bar{x}_2 \end{bmatrix} = \frac{1}{24} \begin{bmatrix} 3 & -6 \\ -6 & 20 \end{bmatrix} \begin{bmatrix} 17 \\ 6.5 \end{bmatrix} = \begin{bmatrix} 1/2 \\ 7/6 \end{bmatrix}.$$

4. L^1 -Regression

Just as the median gives a more robust estimate of the “average value” of a collection of numbers than the mean, L^1 -regression is less sensitive to outliers than least squares regression is. It is defined by minimizing the L^1 -norm of the deviation vector in (12.1). That is, the problem is to find $\hat{x}$ as follows:

$$\hat{x} = \operatorname{argmin}_x \|b - Ax\|_1.$$

Unlike for least squares regression, there is no explicit formula for the solution to the L^1 -regression problem. However, the problem can be reformulated as a linear programming problem. Indeed, it is easy to see that the L^1 -regression problem,

$$\text{minimize } \sum_i \left| b_i - \sum_j a_{ij} x_j \right|,$$

can be rewritten as

$$\begin{array}{ll} \text{minimize} & \sum_i t_i \\ \text{subject to} & t_i - \left| b_i - \sum_j a_{ij} x_j \right| = 0, \quad i = 1, 2, \dots, m, \end{array}$$

which is equivalent to the following linear programming problem:

$$(12.3) \quad \begin{array}{ll} \text{minimize} & \sum_i t_i \\ \text{subject to} & -t_i \leq b_i - \sum_j a_{ij} x_j \leq t_i, \quad i = 1, 2, \dots, m. \end{array}$$

Hence, to solve the L^1 -regression problem, it suffices to solve this linear programming problem. In the next section, we shall present an alternative algorithm for computing the solution to an L^1 -regression problem.

Example. Returning to the example of the last section, the L^1 -regression problem is solved by finding the optimal solution to the following linear programming problem:

$$\begin{array}{llll}
 \text{minimize} & & t_1 + t_2 + t_3 & \\
 \text{subject to} & -x_2 - t_1 & & \leq -1 \\
 & -2x_1 - x_2 & -t_2 & \leq -2.5 \\
 & -4x_1 - x_2 & & -t_3 \leq -3 \\
 & & x_2 - t_1 & \leq 1 \\
 & 2x_1 + x_2 & -t_2 & \leq 2.5 \\
 & 4x_1 + x_2 & & -t_3 \leq 3 \\
 & & & t_1, t_2, t_3 \geq 0.
 \end{array}$$

The solution to this linear programming problem is

$$\hat{x} = \begin{bmatrix} 0.5 \\ 1 \end{bmatrix},$$

which clearly indicates that the point $(2, 2.5)$ is viewed by the L^1 -regression as an outlier, since the regression line passes exactly through the other two points.

5. Iteratively Reweighted Least Squares

Even though calculus cannot be used to obtain an explicit formula for the solution to the L^1 -regression problem, it can be used to obtain an iterative procedure that, when properly initialized, converges to the solution of the L^1 -regression problem. The resulting iterative process is called *iteratively reweighted least squares*. In this section, we briefly discuss this method. We start by considering the objective function for L^1 -regression:

$$\begin{aligned}
 f(x) &= \|b - Ax\|_1 \\
 &= \sum_i \left| b_i - \sum_j a_{ij}x_j \right|.
 \end{aligned}$$

Differentiating this objective function is a problem, since it involves absolute values. However, the absolute value function

$$g(z) = |z|$$

is differentiable everywhere except at one point: $z = 0$. Furthermore, we can use the following simple formula for the derivative, where it exists:

$$g'(z) = \frac{z}{|z|}.$$

Using this formula to differentiate f with respect to each variable, and setting the derivatives to zero, we get the following equations for critical points:

$$(12.4) \quad \frac{\partial f}{\partial x_k} = \sum_i \frac{b_i - \sum_j a_{ij}x_j}{|b_i - \sum_j a_{ij}x_j|} (-a_{ik}) = 0, \quad k = 1, 2, \dots, n.$$

If we introduce the following shorthand notation for the deviations,

$$\epsilon_i(x) = \left| b_i - \sum_j a_{ij}x_j \right|,$$

we see that we can rewrite (12.4) as

$$\sum_i \frac{a_{ik}b_i}{\epsilon_i(x)} = \sum_i \sum_j \frac{a_{ik}a_{ij}x_j}{\epsilon_i(x)}, \quad k = 1, 2, \dots, n.$$

Now, if we let E_x denote the diagonal matrix containing the vector $\epsilon(x)$ on the diagonal, we can write these equations in matrix notation as follows:

$$A^T E_x^{-1} b = A^T E_x^{-1} A x.$$

This equation cannot be solved for x as we were able to do in L^2 -regression because of the dependence of the diagonal matrix on x . But let us rearrange this system of equations by multiplying both sides by the inverse of $A^T E_x^{-1} A$. The result is

$$x = (A^T E_x^{-1} A)^{-1} A^T E_x^{-1} b.$$

This formula suggests an iterative scheme that hopefully converges to a solution. Indeed, we start by initializing x^0 arbitrarily and then use the above formula to successively compute new approximations. If we let x^k denote the approximation at the k th iteration, then the update formula can be expressed as

$$x^{k+1} = (A^T E_{x^k}^{-1} A)^{-1} A^T E_{x^k}^{-1} b.$$

Assuming only that the matrix inverse exists at every iteration, one can show that this iteration scheme converges to a solution to the L^1 -regression problem.

6. An Example: How Fast Is The Simplex Method?

In Chapter 4, we discussed the worst-case behavior of the simplex method and studied the Klee–Minty problem that achieves the worst case. We also discussed the importance of empirical studies of algorithm performance. In this section, we shall introduce a model that allows us to summarize the results of these empirical studies.

We wish to relate the number of simplex iterations T required to solve a linear programming problem to the number of constraints m and/or the number of variables n in the problem (or some combination of the two). As any statistician will report, the first step is to introduce an appropriate model.³ Hence, we begin by asking: *how many iterations, on average, do we expect the simplex method to take if the problem has m constraints and n variables?* To propose an answer to this question, consider the initial dictionary associated with a given problem. This dictionary involves m values, x_B^* , for the primal basic variables, and n values, y_N^* , for the dual nonbasic variables. We would like each of these $m + n$ variables to have nonnegative values, since that would indicate optimality. If we assume that the initial dictionary is nondegenerate, then one would expect on the average that $(m + n)/2$ of the values would be positive and the remaining $(m + n)/2$ values would be negative.

Now let us look at the dynamics of the simplex method. Each iteration focuses on exactly one of the negative values. Suppose, for the sake of discussion, that the negative value corresponds to a dual nonbasic variable, that is, one of the coefficients in the objective row of the dictionary. Then the simplex method selects the corresponding primal nonbasic variable to enter the basis, and a leaving variable is chosen by a ratio test. After the pivot, the variable that exited now appears as a nonbasic variable in the same position that the entering variable held before. Furthermore, the coefficient on this variable is guaranteed to be positive (since we have assumed nondegeneracy). Hence, the effect of one pivot of the simplex method is to correct the sign of one of the negative values from the list of $m + n$ values of interest. Of course, the pivot also affects all the other values, but there seems no reason to assume that the situation relative to them will have any tendency to get better or worse, on the average. Therefore, we can think of the simplex method as statistically reducing the number of negative values by one at each iteration.

Since we expect on the average that an initial dictionary will have $(m + n)/2$ negative values, it follows that the simplex method should take $(m + n)/2$ iterations, on average. Of course, these expectations are predicated on the assumption that degenerate dictionaries do not arise. As we saw in Section 7.2, the self-dual simplex method initialized with random perturbations will, with probability one, never encounter a degenerate dictionary. Hence, we hypothesize that this variant of the simplex method will, on average, take $(m + n)/2$ iterations. It is important to note the main point of our hypothesis; namely, that the number of iterations is *linear* in $m + n$ as opposed, say, to quadratic or cubic.

³In the social sciences, a fundamental difficulty is the lack of specific arguments validating the appropriateness of the models commonly introduced.

We can test our hypothesis by first supposing that T can be approximated by a function of the form

$$2^\alpha (m+n)^\beta$$

for a pair of real numbers α and β . Our goal then is to find the value for these parameters that best fits the data obtained from a set of empirical observations. (We have written the leading constant as 2^α simply for symmetry with the other factor—there is no fundamental need to do this.) This multiplicative representation of the number of iterations can be converted into an additive (in α and β) representation by taking logarithms. Introducing an ϵ to represent the difference between the model's prediction and the true number of iterations, we see that the model can be written as

$$\log T = \alpha \log 2 + \beta \log(m+n) + \epsilon.$$

Now, suppose that several observations are made. Using subscripts to distinguish the various observations, we get the following equations:

$$\begin{bmatrix} \log T_1 \\ \log T_2 \\ \vdots \\ \log T_k \end{bmatrix} = \begin{bmatrix} \log 2 & \log(m_1 + n_1) \\ \log 2 & \log(m_2 + n_2) \\ \vdots & \vdots \\ \log 2 & \log(m_k + n_k) \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} + \begin{bmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_k \end{bmatrix}.$$

If we let b denote the vector on the left, A the matrix on the right, x the vector multiplied by A , and ϵ the vector of deviations, then the model can be expressed as

$$b = Ax + \epsilon,$$

where A and b are given. As we have seen, this is just a regression model, which we can solve as an L^1 -regression or as an L^2 -regression.

Given real data, we shall solve this model both ways. Table 12.1 shows specific data obtained by running the self-dual simplex method described in Chapter 7 (with randomized initial perturbations) against most of the problems in a standard suite of test problems (called the NETLIB suite (Gay 1985)). Some problems were too big to run on the workstation used for this experiment, and others were formulated with free variables that the code was not equipped to handle.

Using (12.2) to solve the problem as an L^2 -regression, we get

$$\begin{bmatrix} \bar{\alpha} \\ \bar{\beta} \end{bmatrix} = \begin{bmatrix} -1.03561 \\ 1.05152 \end{bmatrix}.$$

Or, in other words,

$$T \approx 0.488(m+n)^{1.052}.$$

This is amazingly close to our hypothesized formula, $(m+n)/2$. Figure 12.3 shows a log-log plot of T vs. $m+n$ with the L^2 -regression line drawn through it. It is clear from this graph that a straight line (in the log-log plot) is a good model for fitting this data.

Name	m	n	Iters	Name	m	n	Iters
25fv47	777	1,545	5,089	nesm	646	2,740	5,829
80bau3b	2,021	9,195	10,514	recipe	74	136	80
adlittle	53	96	141	sc105	104	103	92
afiro	25	32	16	sc205	203	202	191
agg2	481	301	204	sc50a	49	48	46
agg3	481	301	193	sc50b	48	48	53
bandm	224	379	1,139	scagr25	347	499	1,336
beaconfd	111	172	113	scagr7	95	139	339
blend	72	83	117	scfxm1	282	439	531
bnl1	564	1,113	2,580	scfxm2	564	878	1,197
bnl2	1,874	3,134	6,381	scfxm3	846	1,317	1,886
boeing1	298	373	619	scorpion	292	331	411
boeing2	125	143	168	scrs8	447	1,131	783
bore3d	138	188	227	scsd1	77	760	172
brandy	123	205	585	scsd6	147	1,350	494
czprob	689	2,770	2,635	scsd8	397	2,750	1,548
d6cube	403	6,183	5,883	sctap1	284	480	643
degen2	444	534	1,421	sctap2	1,033	1,880	1,037
degen3	1,503	1,818	6,398	sctap3	1,408	2,480	1,339
e226	162	260	598	seba	449	896	766
etamacro	334	542	1,580	share1b	107	217	404
fffff800	476	817	1,029	share2b	93	79	189
finnis	398	541	680	shell	487	1,476	1,155
fit1d	24	1,026	925	ship04l	317	1,915	597
fit1p	627	1,677	15,284	ship04s	241	1,291	560
forplan	133	415	576	ship08l	520	3,149	1,091
ganges	1,121	1,493	2,716	ship08s	326	1,632	897
greenbea	1,948	4,131	21,476	ship12l	687	4,224	1,654
grow15	300	645	681	ship12s	417	1,996	1,360
grow22	440	946	999	sierra	1,212	2,016	793
grow7	140	301	322	standata	301	1,038	74
israel	163	142	209	standmps	409	1,038	295
kb2	43	41	63	stocfor1	98	100	81
lotfi	134	300	242	stocfor2	2,129	2,015	2,127
maros	680	1,062	2,998				

TABLE 12.1. Number of iterations for the self-dual simplex method.

Using (12.3) to solving the problem, we get

$$\begin{bmatrix} \bar{\alpha} \\ \bar{\beta} \end{bmatrix} = \begin{bmatrix} -0.9508 \\ 1.0491 \end{bmatrix}.$$

In other words,

$$T \approx 0.517(m+n)^{1.049}.$$

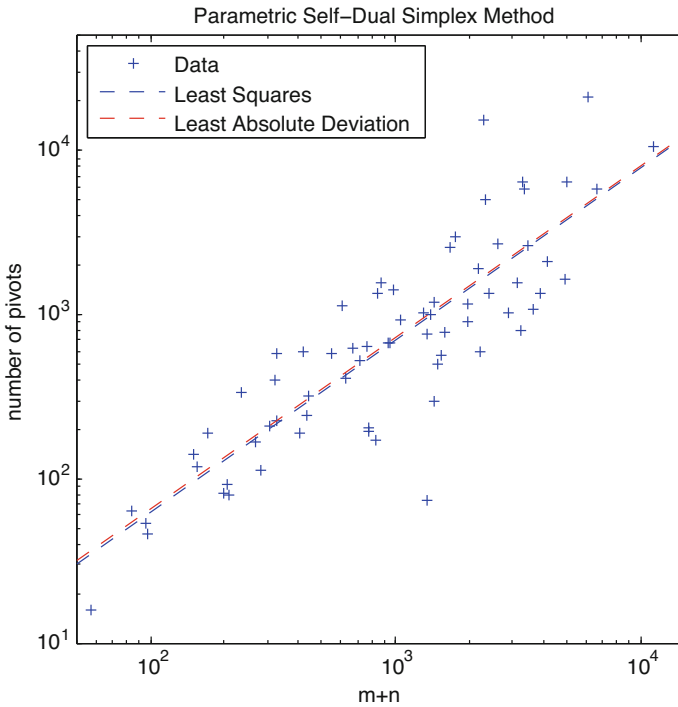


FIGURE 12.3. A log-log plot of T vs. $m + n$ and the L^1 and L^2 regression lines.

The fact that this regression formula agrees closely with the L^2 -regression indicates that the data set contains no outliers. In Section 12.6.1, we will consider randomly generated problems and see at least one example where the L^1 and L^2 regression lines differ significantly.

6.1. Random Problems. Now, let us consider random problems generated in a manner similar to the way we did it back in Chapter 4. We do, however, introduce some changes. First of all, the problems in Chapter 4 were generated in such a manner as to guarantee primal feasibility but dual feasibility was left to chance—that is, many (half) of the problems were unbounded. The problems we wish to consider now will be assumed to have optimal solutions (real-world problems are often, but not always, known to have an optimal solution because of the underlying physical model and therefore primal or dual infeasibility is often an indicator of data and/or modeling errors). To guarantee the existence of an optimal solution, we generate random optimal primal and dual solutions and associated random optimal slack/surplus variables. Here's the MATLAB code for that:

```

x = round(sigma*rand(n,1)).*(rand(n,1)>0.5);
y = round(sigma*rand(1,m)).*(rand(1,m)>0.5);
z = round(sigma*rand(1,n)).*(rand(1,n)>0.5);
w = round(sigma*rand(m,1)).*(rand(m,1)>0.5);

```

(as in Chapter 4, σ is a constant initialized to 10). We then define A , b , and c as problem data consistent with these optimal solutions:

```

A = round(sigma*(randn(m,n))).*(rand(m,n)>0.5);
b = A*x + w;
c = y*A - z;

```

Note that we have made one other key change from before—we have randomly forced about half of the optimal values of the variables and about half of the constraint matrix coefficients to be zero. This change makes the problems slightly more realistic as real-world problems often have much sparsity.

Next, we need to initialize a right-hand side perturbation and an objective function perturbation:

```

b0 = rand(m,1);
c0 = -rand(1,n);

```

There are just three relatively simple changes to the code defining the simplex method itself to convert it from a primal-simplex algorithm to the parametric self-dual method. The first is to change the line of code that checks if the problem has been solved. Before, we only needed to check if all of the objective coefficients had become negative (dual feasibility) because primal feasibility was built in. Now, we have to check both primal and dual feasibility:

```

while max(c) > eps || min(b) < -eps,

```

Secondly, the choice of enter/leaving variables must be updated as it is now based on minimizing the perturbation parameter:

```

[mu_col, col] = max( (-c./c0).*(c0<-eps));
[mu_row, row] = max( (-b./b0).*(b0> eps));
if mu_col >= mu_row,
    mu = mu_col;
    Acol = A(:,col);
    [t, row] = max(-Acol./(b+mu*b0));
else
    mu = mu_row;
    Arow = A(row,:);
    [s, col] = max(-Arow./(c+mu*c0));
end

```

Finally, as part of every pivot we have to update b_0 and c_0 :

```

brow = b0(row);
b0 = b0 - brow*Ac0l/a;
b0(row) = -brow/a;

ccol = c0(col);
c0 = c0 - ccol*Ar0w/a;
c0(col) = ccol/a;

```

The code was run 1,000 times. Figure 12.4 shows the number of pivots plotted against the sum $m + n$. Just as we saw with the primal simplex method in Chapter 4, $m + n$ does not seem to be a good measure of problem size as many problems of a given size solve much more quickly than the more typical cases. Hence, there are a number of “outliers.” Overlaid on the scatter plot are the L^1 and L^2 regression lines. While neither regression line follows what appears to be an upper line of points that seems to dominate the results, the L^1 is closer to that than is the L^2 line.

The result of the L^1 -regression is

$$T \approx e^{-0.722} e^{1.12 \log(m+n)} = 0.486(m+n)^{1.12}.$$

The result of the L^2 -regression is

$$T \approx e^{-0.606} e^{1.05 \log(m+n)} = 0.546(m+n)^{1.05}.$$

Finally, as in Chapter 4, $\min(m, n)$ is a better measure of problem size for these randomly generated problems. Figure 12.5 shows the same data plotted against $\min(m, n)$.

In this case, both regression lines are about the same:

$$T \approx e^{-0.2} e^{1.46 \log(\min(m,n))} = 0.8 \min(m, n)^{1.46}.$$

7. Character Recognition: A Support Vector Machine

In today’s world, algorithms for speech recognition have made computers seem almost human—thank you Siri and Alexa! As cool as that is, let us focus on something a little bit easier to explain... handwritten character recognition. To keep the story as simple as possible, let us assume that you (or someone you know) has written some letters of the alphabet on a piece of paper, and you then scan an image of them into a computer, convert the image to an array of “brightnesses,” and employ an algorithm to determine what the letters are. In this section, we will focus on the last step in this process. In the computer world, this is called *optical character recognition* (OCR).

The idea behind OCR is to collect hundreds, perhaps thousands, maybe even millions, of handwriting samples where it is known what the written letters are. These samples are called the “training set.” Using a training set, the algorithm we will describe,

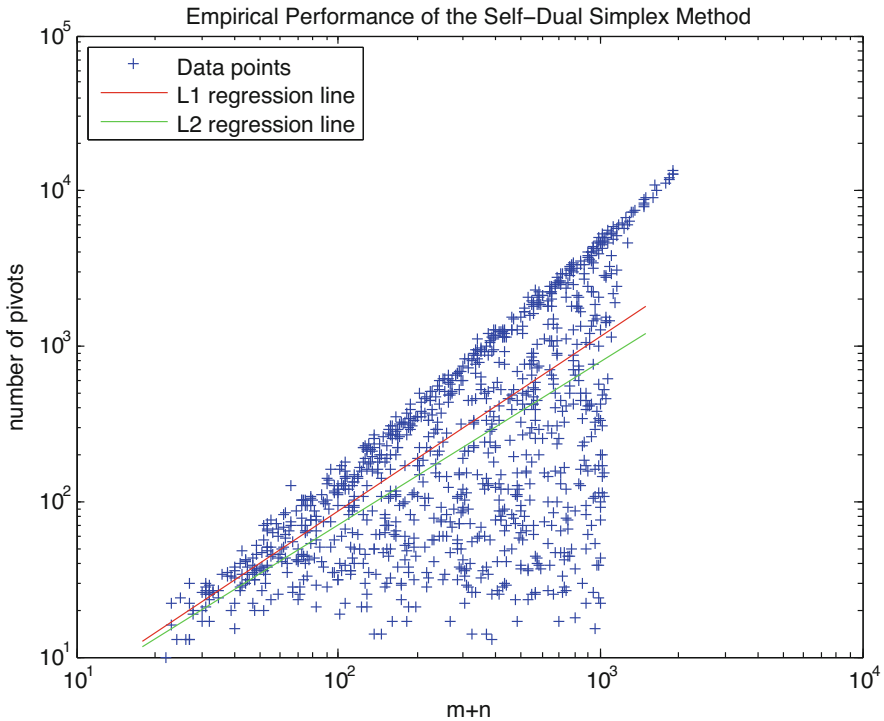


FIGURE 12.4. The parametric self-dual simplex method was used to solve 1,000 problems known to have an optimal solution. Shown here is a log–log plot showing the number of pivots required to reach optimality plotted against $m + n$. Also shown are the L^1 and L^2 regression lines.

which is called a *support vector machine (SVM)*, represents these data in some high-dimensional space and, for each newly written letter you wish to identify, the algorithm will use the training set to determine which letter it most likely is and report that letter as the answer.

Before addressing the problem of detecting all letters from an alphabet, let us consider the simpler problem of just detecting whether a letter is a Y (for “yes”) or an N (for “no”). Associated with each sample in the training set, we have a vector x . Eventually, this vector will contain brightness values for each pixel in our scanned image of the letter and our scans will be setup to produce 12×12 pixel images so that the vector

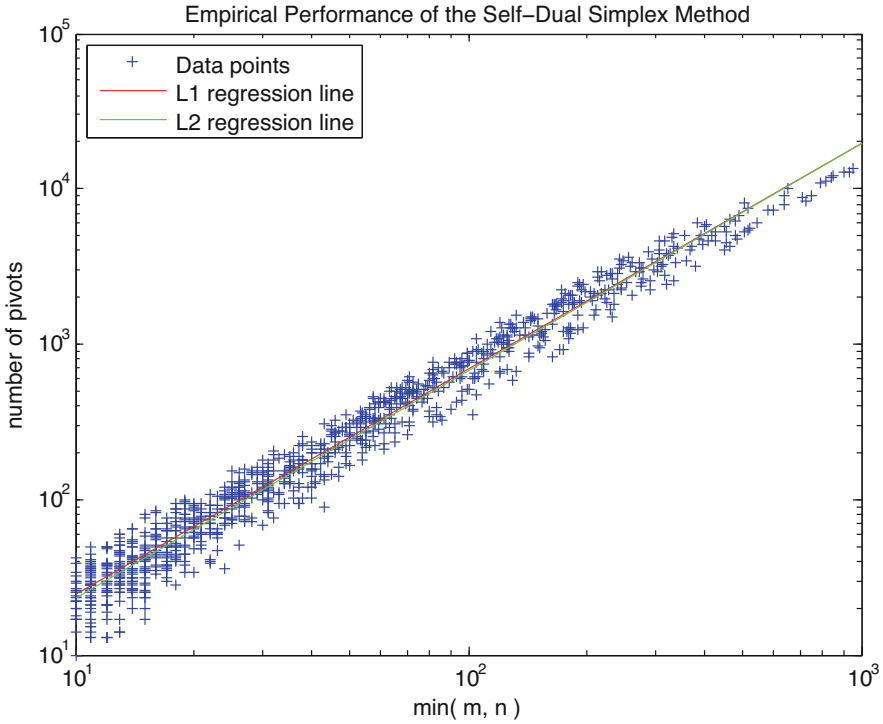


FIGURE 12.5. The parametric self-dual simplex method was used to solve 1,000 problems known to have an optimal solution. Shown here is a log–log plot showing the number of pivots required to reach optimality plotted against $\min(m, n)$. In this case, the L^1 and L^2 regression lines are almost exactly on top of each other.

x will have 144 entries (and will more conveniently be stored as a matrix instead of as a vector). But, before we get into the real data, let us just imagine a scenario where the x -vectors are only two-dimensional so that we can make some pictures to illustrate the SVM idea.

Figure 12.6 shows a plot of 25 vectors associated with the letter “Y” and 15 vectors associated with the letter “N”. Our aim is to find the equation of a line, $a_1x_1 + a_2x_2 = b$, such that most/all of the Y’s in the training set fall on one side of the line and most/all of the N’s fall on the other side. Let $x = (x_1, x_2)$ denote a generic 2-vector. If

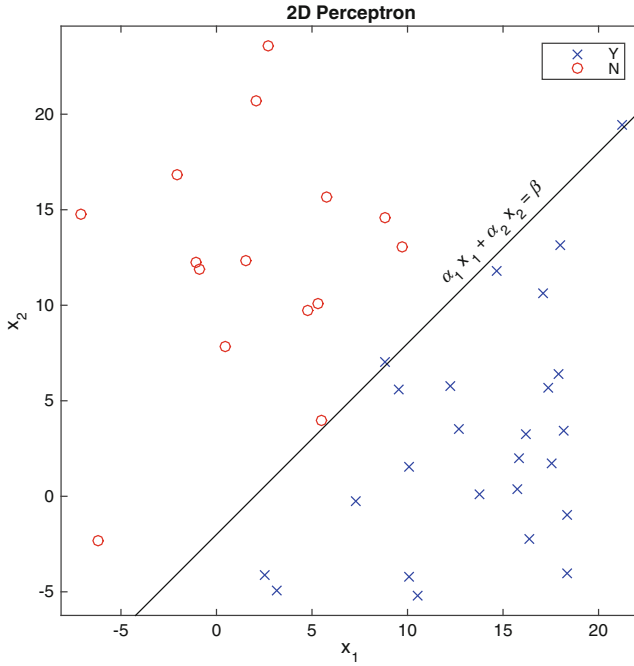


FIGURE 12.6. Vectors from the training set corresponding to 'Y' are shown as blue $\times$'s, whereas vectors corresponding to 'N' are shown as red $\circ$'s. The (imperfect) line of separation is $a_1x_1 + a_2x_2 = b$.

$a_1x_1 + a_2x_2 - b = 0$, then x must, by definition, lie on the line. If $a_1x_1 + a_2x_2 - b > 0$, then x lies on one side of the line and if $a_1x_1 + a_2x_2 - b < 0$, it lies on the other side. So, if we label our 40 training points (25 Y's and 15 N's),

$$x^i = \begin{bmatrix} x_1^i \\ x_2^i \end{bmatrix}, \quad i = 1, 2, \dots, 40,$$

$$\text{label}(i) = \text{'Y' or 'N'}, \quad i = 1, 2, \dots, 40,$$

then we could require that our training sample satisfies these inequalities:

$$\begin{aligned} a_1x_1^i + a_2x_2^i - b &> 0, & \text{if } \text{label}(i) = \text{'Y'}, \\ a_1x_1^i + a_2x_2^i - b &< 0, & \text{if } \text{label}(i) = \text{'N'}. \end{aligned}$$

As the plot in Figure 12.6 shows, some of the training points will be far from the separating line, while others might be close to it and some might even be on the wrong side. Let ε_i denote “how far” sample point i is from the line with positive values meaning we are on the “correct” side:

$$\varepsilon_i = \begin{cases} a_1 x_1^i + a_2 x_2^i - b, & \text{if label}(i) = 'Y', \\ -(a_1 x_1^i + a_2 x_2^i - b), & \text{if label}(i) = 'N'. \end{cases}$$

Our problem can now be described as: find the coefficients a_1 and a_2 and “intercept” b so as to maximize the minimum value of the ε_i ’s.

There remains one issue. Given a choice of the a ’s and the b , we could multiply these coefficients by an arbitrary large number and it would still represent exactly the same line. But, the ε ’s would get much bigger. So, if we are going to optimize by maximizing the minimum of the ε_i ’s, we need to impose some sort of scale constraint on the choice of the a ’s and/or b . Since the vector $a = [a_1, a_2]^T$ is the *normal* vector, it seems quite sensible to impose the extra constraint that a be a unit vector:

$$a_1^2 + a_2^2 = 1.$$

In fact, imposing this constraint makes the ε ’s into true Euclidean distances. Unfortunately, including this constraint in our optimization problem makes the problem both nonlinear and non-convex, which is not a good idea. But, since we are maximizing the sum of the ε ’s and since asking for large ε ’s tends to push us toward large a vectors, we can replace the equality constraint with an inequality constraint:

$$(12.5) \quad a_1^2 + a_2^2 \leq 1.$$

This constraint is still nonlinear but it is convex and being convex versus not is the key distinction between being a tractable optimization problem versus an intractable one.

We can now fully describe our SVM model:

$$\begin{aligned} & \underset{\varepsilon_i \text{'s}, a_1, a_2, b}{\text{maximize}} && \min_i \varepsilon_i \\ & \text{subject to:} && \varepsilon_i = a_1 x_1^i + a_2 x_2^i - b, && \text{if label}(i) = 'Y', \\ & && \varepsilon_i = -(a_1 x_1^i + a_2 x_2^i - b), && \text{if label}(i) = 'N', \\ & && a_1^2 + a_2^2 \leq 1. \end{aligned}$$

As written, this problem is not a linear optimization problem. There are a few different ways that we can use to make a linear formulation. For example, we can replace the constraint on the sum of squares with simple upper and lower bounds of 1 on each of these two variables:

$$\begin{aligned} -1 &\leq a_1 \leq 1 \\ -1 &\leq a_2 \leq 1. \end{aligned}$$

And, we can introduce a new variable, let us call it δ that represents the minimum of the ε_i 's. Of course, instead of imposing the constraint that

$$\delta = \min_i \varepsilon_i$$

we instead use a bunch of bound constraints:

$$\delta \leq \varepsilon_i \quad i = 1, 2, \dots, 40.$$

These constraints simply guarantee that δ is not larger than the minimum. It could be strictly smaller than the minimum. But, the objective function is trying to maximize δ . At optimality, δ will be equal to the minimum of the ε_i 's. So, here is a linear variant of the OCR problem:

$$\begin{aligned} &\underset{\varepsilon_i \text{'s}, \delta, a_1, a_2, b}{\text{maximize}} && \delta \\ \text{subject to:} &&& \varepsilon_i = a_1 x_1^i + a_2 x_2^i - b, && \text{if label}(i) = \text{'Y'}, \\ (12.6) &&& \varepsilon_i = -(a_1 x_1^i + a_2 x_2^i - b), && \text{if label}(i) = \text{'N'}, \\ &&& \delta \leq \varepsilon_i, && i = 1, 2, \dots, 40, \\ &&& -1 \leq a_i \leq 1 && i = 1, 2. \end{aligned}$$

The conceptual explanation given above assumed that the data points were two-dimensional vectors. For the actual character recognition problem we assume that the data points are, say, 12×12 arrays. Thought of as a vector, each data point is a 144-dimensional vector. It is hard to draw a picture of that, but the basic idea is exactly the same as in the two-dimensional case. Figure 12.7 shows what a few of the “Y” and “N” data arrays might look like.

It is easy to extend the idea presented above to develop a tool than can identify the letters in an alphabet consisting of, say, 26 letters. Instead of solving just one SVM optimization problem, we will solve 26 problems. The first problem will set the “labels” as “A” versus “not-A,” the second problem will set the labels as “B” versus “not-B,” etc. Then, when testing a given letter against the 26 possibilities, one hopes that 25 will say not-this-letter and one will say yes you got it. Of course, that can be wishful thinking and there might be no obvious winning letter or there might be multiple possible winners. In any case, the letter to pick is the one that gave the largest value to the objective function for that choice.

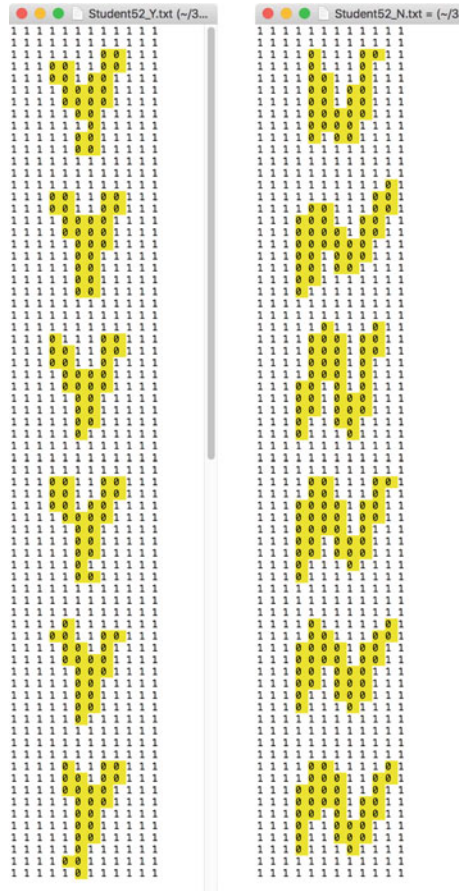


FIGURE 12.7. Sample data showing the letter “Y” written six times and the letter “N” written six times. The ones are highlighted in yellow just to make them easier to see.

Exercises

- 12.1** Find the L^2 -regression line for the data shown in Figure 12.8.
- 12.2** Find the L^1 -regression line for the data shown in Figure 12.8.
- 12.3** *Midrange*. Given a sorted set of real numbers, $\{b_1, b_2, \dots, b_m\}$, show that the midrange, $\tilde{x} = (b_1 + b_m)/2$, minimizes the maximum deviation from the set of observations. That is,

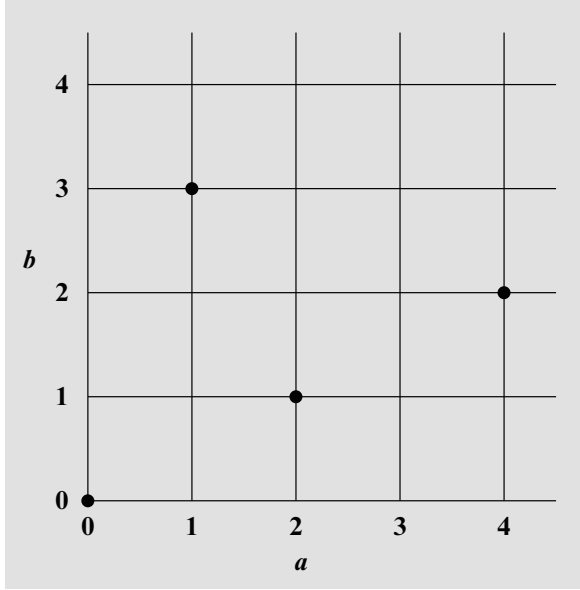


FIGURE 12.8. Four data points for a linear regression.

$$\frac{1}{2}(b_1 + b_m) = \operatorname{argmin}_{x \in \mathbb{R}} \max_i |x - b_i|.$$

12.4 Centroid. Given a set of points $\{b_1, b_2, \dots, b_m\}$ on the plane $\mathbb{R}^2$, show that the centroid

$$\bar{x} = \frac{1}{m} \sum_{i=1}^m b_i$$

minimizes the sum of the squares of the distance to each point in the set. That is, $\bar{x}$ solves the following optimization problem:

$$\text{minimize } \sum_{i=1}^m \|x - b_i\|_2^2$$

Note: Each data point b_i is a vector in $\mathbb{R}^2$ whose components are denoted, say, by b_{i1} and b_{i2} , and, as usual, the subscript 2 on the norm denotes the Euclidean norm. Hence,

$$\|x - b_i\|_2 = \sqrt{(x_1 - b_{i1})^2 + (x_2 - b_{i2})^2}.$$

12.5 Facility Location. A common problem is to determine where to locate a facility so that the distance from its customers is minimized. That is, given

Jan	390	May	310	Sep	550
Feb	420	Jun	590	Oct	360
Mar	340	Jul	340	Nov	420
Apr	320	Aug	580	Dec	600

TABLE 12.2. Projected labor hours by month.

a set of points $\{b_1, b_2, \dots, b_m\}$ on the plane $\mathbb{R}^2$, the problem is to find $\hat{x} = (\hat{x}_1, \hat{x}_2)$ that solves the following optimization problem:

$$\text{minimize } \sum_{i=1}^m \|x - b_i\|_2.$$

As for L^1 -regression, there is no explicit formula for $\hat{x}$, but an iterative scheme can be derived along the same lines as in Section 12.5. Derive an explicit formula for this iteration scheme.

- 12.6** *A Simple Steiner Tree.* Suppose there are only three customers in the facility location problem of the previous exercise. Suppose that the triangle formed by b_1 , b_2 , and b_3 has no angles greater than 120° . Show that the solution $\hat{x}$ to the facility location problem is the unique point in the triangle from whose perspective the three customers are 120° apart from each other. What is the solution if one of the angles, say, at vertex b_1 , is more than 120° ?
- 12.7** *Sales Force Planning.* A distributor of office equipment finds that the business has seasonal peaks and valleys. The company uses two types of sales persons: (a) regular employees who are employed year-round and cost the company \$17.50/h (fully loaded for benefits and taxes) and (b) temporary employees supplied by an outside agency at a cost of \$25/h. Projections for the number of hours of labor by month for the following year are shown in Table 12.2. Let a_i denote the number of hours of labor needed for month i and let x denote the number of hours per month of labor that will be handled by regular employees. To minimize total labor costs, one needs to solve the following optimization problem:

$$\text{minimize } \sum_i (25 \max(a_i - x, 0) + 17.50x).$$

- Show how to reformulate this problem as a linear programming problem.
- Solve the problem for the specific data given above.
- Use calculus to find a formula giving the optimal value for x .

12.8 Acceleration Due to Gravity. The law of gravity from classical physics says that an object dropped from a tall building will, in the absence of air resistance, have a constant rate of acceleration g so that the height x , as a function of time t , is given by

$$x(t) = -\frac{1}{2}gt^2.$$

Unfortunately, the effects of air resistance cannot be ignored. To include them, we assume that the object experiences a retarding force that is directly proportional to its speed. Letting $v(t)$ denote the velocity of the object at time t , the equations that describe the motion are then given by

$$\begin{aligned} x'(t) &= v(t), & t > 0, & & x(0) &= 0, \\ v'(t) &= -g - fv(t), & t > 0, & & v(0) &= 0 \end{aligned}$$

(f is the unknown constant of proportionality from the air resistance). These equations can be solved explicitly for x as a function of t :

$$\begin{aligned} x(t) &= -\frac{g}{f^2} (e^{-ft} - 1 + ft) \\ v(t) &= -\frac{g}{f} (1 - e^{-ft}). \end{aligned}$$

It is clear from the equation for the velocity that the *terminal velocity* is g/f . It would be nice to be able to compute g by measuring this velocity, but this is not possible, since the terminal velocity involves both f and g . However, we can use the formula for $x(t)$ to get a two-parameter model from which we can compute both f and g . Indeed, if we assume that all measurements are taken after terminal velocity has been “reached” (i.e., when e^{-ft} is much smaller than 1), then we can write a simple linear expression relating position to time:

$$x = \frac{g}{f^2} - \frac{g}{f}t.$$

Now, in our experiments we shall set values of x (corresponding to specific positions below the drop point) and measure the time at which the object passes these positions. Since we prefer to write regression models with the observed variable expressed as a linear function of the control variables, let us rearrange the above expression so that t appears as a function of x :

$$t = \frac{1}{f} - \frac{f}{g}x.$$

Using this regression model and the data shown in Table 12.3, do an L^2 -regression to compute estimates for $1/f$ and $-f/g$. From these estimates derive an estimate for g . If you have access to linear programming software, solve the problem using an L^1 -regression and compare your answers.

Obs. number	Position (m)	Time (s)
1	-10	3.72
2	-20	7.06
3	-30	10.46
4	-10	3.71
5	-20	7.00
6	-30	10.48
7	-10	3.67
8	-20	7.08
9	-30	10.33

TABLE 12.3. Time at which a falling object passes certain points.

12.9 *Iteratively Reweighted Least Squares.* Show that the sequence of iterates in the iteratively reweighted least squares algorithm produces a monotonically decreasing sequence of objective function values by filling in the details in the following outline. First, recall that the objective function for L^1 -regression is given by

$$f(x) = \|b - Ax\|_1 = \sum_{i=1}^m \epsilon_i(x),$$

where

$$\epsilon_i(x) = \left| b_i - \sum_{j=1}^n a_{ij}x_j \right|.$$

Also, the function that defines the iterative scheme is given by

$$T(x) = (A^T E_x^{-1} A)^{-1} A^T E_x^{-1} b,$$

where E_x denotes the diagonal matrix with the vector $\epsilon(x)$ on its diagonal. Our aim is to show that

$$f(T(x)) < f(x).$$

In order to prove this inequality, let

$$g_x(z) = \sum_{i=1}^m \frac{\epsilon_i^2(z)}{\epsilon_i(x)} = \|E_x^{-1/2}(b - Az)\|_2^2.$$

- (a) Use calculus to show that, for each x , $T(x)$ is a global minimum of g_x .
- (b) Show that $g_x(x) = f(x)$.
- (c) By writing

$$\epsilon_i(T(x)) = \epsilon_i(x) + (\epsilon_i(T(x)) - \epsilon_i(x))$$

and then substituting the right-hand expression into the definition of $g_x(T(x))$, show that

$$g_x(T(x)) \geq 2f(T(x)) - f(x).$$

- (d) Combine the three steps above to finish the proof.

12.10 In our study of means and medians, we showed that the median of a collection of numbers, $b_1, b_2, \dots, b_n$, is the number $\hat{x}$ that minimizes $\sum_j |x - b_j|$. Let μ be a real parameter.

- (a) Give a statistical interpretation to the following optimization problem:

$$\text{minimize } \sum_j (|x - b_j| + \mu(x - b_j)).$$

Hint: the special cases $\mu = 0, \pm 1/2, \pm 1$ might help clarify the general situation.

- (b) Express the above problem as a linear programming problem.
- (c) The parametric simplex method can be used to solve families of linear programming problems indexed by a parameter μ (such as we have here). Starting at $\mu = \infty$ and proceeding to $\mu = -\infty$ one solves all of the linear programs with just a finite number of pivots. Use the parametric simplex method to solve the problems of the previous part in the case where $n = 4$ and $b_1 = 1, b_2 = 2, b_3 = 4$, and $b_4 = 8$.
- (d) Now consider the general case. Write down the dictionary that appears in the k -th iteration and show by induction that it is correct.

12.11 Show that the L^∞ -norm is just the maximum of the absolute values. That is,

$$\lim_{p \rightarrow \infty} \|x\|_p = \max_i |x_i|.$$

12.12 Here is a webpage that implements the optical character recognition algorithm shown in (12.6)

vanderbei.princeton.edu/JAVA/SVM/perceptron_linf.html

- (a) Use your computer's mouse or touchpad to record a message and then use the machine to decipher your message.
- (b) Use the *Update Training Set* part of the page to input the alphabet multiple times using your own handwriting. Then, redetermine the message and see if the result is better or worse than before.

Notes

Gonin and Money (1989) and Dodge (1987) are two references on regression that include discussion of both L^2 and L^1 regression. The standard reference on L^1 regression is Bloomfield and Steiger (1983).

Several researchers, including Smale (1983), Borgwardt (1982), Borgwardt (1987a), Adler and Megiddo (1985), and Todd (1986), have studied the average number of iterations of the simplex method as a function of m and/or n . The model discussed in this chapter is similar to the sign-invariant model introduced by Adler and Berenguer (1981).

Support vector machines were first introduced by Vapnik and his team at AT&T in the 1980s. Here are two early papers on the subject: Cortes and Vapnik (1995) and Boser et al. (1992).